

Line #	Hits	Time	Per Hit	% Time	Line Contents
15					@profile
16					def mult(matrix1, matrix2):
17					# Matrix multiplication
18	1	14	14.0	0.0	if len(matrix1[0]) != len(matrix2):
19					print 'Matrices must be m*n and n*p to multiply!'
20					else:
21	1	4	4.0	0.0	l1=len(matrix1)
22	1	3	3.0	0.0	l2=len(matrix2[0])
23	1	2	2.0	0.0	l3=len(matrix2)
24	1	179901	179901.0	0.0	new_matrix = zero(l1,l2)
25	17	40	2.4	0.0	for i in xrange(l1):
26	8032	16590	2.1	0.1	for j in xrange(l2):
27	4615919	8276282	2.1	38.0	for k in xrange(l3):
28	4607904	13320933	3.3	61.1	new_matrix[i][j] += matrix1[i][k]*matrix2[k][j]
29					return new_matrix

Figure 5: Line_profiler

```
>>>
```

512 function calls in 39.986 seconds

Ordered by: standard name

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.004	0.004	39.986	39.986	<string>:1(<module>)
1	0.034	0.034	0.036	0.036	matrix_mult.py:11(zero)
1	39.946	39.946	39.982	39.982	matrix_mult.py:15(mult)
7	0.000	0.000	0.000	0.000	{len}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_laprof.Profiler' objects}
501	0.003	0.000	0.003	0.000	{range}

Figure 6: Lineopt output

The output you get is shown in Figure 5. Since *line_profiler* lets you see which line is causing the biggest problems, you can now focus on fixing the problem by directly improving the slow code. Let's analyse the output of the line profiler:

- Line #: This is the line number in the file.
- Hits: The number of times that line was executed.
- Time: The total amount of time spent executing the line.
- Per Hit: The average amount of time spent executing the line.
- % Time: The percentage of time spent on that line.
- Line Contents: The actual source code.

Optimisations

Notice how the innermost line '*new_matrix[i][j] += matrix1[i][k]*matrix2[k][j]*' executes for 61 per cent of the total time. This means that optimising this one line can immensely boost your execution speed. Hence, I tried the following optimisation:

```
def mult(matrix1, matrix2):
    # Matrix multiplication
    if len(matrix1[0]) != len(matrix2):
        print 'Matrices must be m*n and n*p to multiply!'
    else:
        new_matrix = matrix(len(matrix1), len(matrix2[0]))
        new_matrix.zero()
        for i in range(len(matrix1)):
            for j in range(len(matrix2[0])):
```

```
                c=0
                for k in range(len(matrix2)):
                    c += matrix1[i][k]*matrix2[k][j]
                new_matrix.mat[i][j]=c
            return new_matrix
```

I tried this small change because *i* and *j* remain constant through the execution of the innermost loop; the time taken in that line is increased due to memory access for a location in memory that stores the array values. Hence, I replaced the array variable with a variable *c*, and assigned the value of *c* to the corresponding array element. Now, the array will be accessed from memory only once rather than *k* times, which slowed down the code drastically, as can be seen in the cProfile output.

As you can see in Figure 6, code execution time has come down to 39 seconds just by this small change!

Arriving at optimum performance involves a lot of trial and error. All the changes you make in your code may or may not work in your favour. Keep experimenting and try to understand what is slowing your code down, and why. Some changes end up slowing down your code, so use profiling tools to give you a clear picture of which areas of your code need your attention for tuning. In the next article in this series, I will cover Cython and PyPy as tools to improve Python's performance. **END** 

By: Sahil Chelaramani

The author is an open source activist, who loves Linux, Python and Android. A third year student at the Goa Engineering College (GEC), he loves to code beyond anything else.